Scripting in Orion Engine

20-10-2026

Overview

Scripting is done via a Scheme-like language, and wired into the engine using OScriptable components.

Alternatives

Python, Lua, and 'roll it yourself' languages were considered, and Lua is still a contender should Lisp not work out. Python was dismissed as too heavy, 'roll it yourself' ruled out because I would like to ship a game *at some point*.

Why?

- Because it felt like the kind of 'hard way'/edgy thing I seem to enjoy subjecting myself too.
- Because I saw this little company Naughty Dog did something similar (GOAL) for Crash Bandicoot and that might've been my favorite PS2 game I owned

Ackshwally...

Yeah I know it's not 'really' Lisp and not 'really' Scheme. You can put a detailed description of why I should be burnt at the stake at `/dev/null`

For now this means no LSP, no debugger, no... anything really. "Custom bindings are cool" and "I did the start of the hard thing" are about all it has going for it, and I've made my peace with that for now. If I wanted comfy and polished tools I'd have grabbed Unity and accepted a splash screen, or Godot and embraced FLOSS. The point of Orion is to *not* be an off the shelf engine, and custom scripting is a line I am drawing to not pull in a library for.

Engine usage

Parsing Lisp is something you could've had to do in a CS degree course. I didn't have to quite do it exactly, but our spreadsheet assignment in CS 3500 had a similar enough requirement, this felt like it'd be a no brainer.

I was only a little bit wrong.

Loading

0. Scripts for now have to be located in /sandbox/assets/scripts/.oscr. `.oscr` for now is 'orion script'. Super clever

1. Scripts are read character by character from the source file, into a buffer (`std::string`).
2. The buffer is then 'tokenized'.
    - I don't love the term (*thanks Claude*) but naming things is hard and it's pretty accurate to what it's doing. Tokens in this case are each 'unit'. Map to a `Value`, but that's a runtime invocation concept, not a data holding concept.
3. Tokens then get parsed into our 'Call Tree'. It's in quotes because I wanted to call it a Call Stack which feels less made up, but stack implies up and down only and our tree has a left to right part too.
4. Call Tree gets stored as a vector of Nodes, and then at execution time the vector is iterated over, with each Node tree getting traversed. This gives us both "breadth and depth" script handling at the cost of single digits worth of type complexity.

```
; line numbers are made up and pretend they aren't there, but it'll help documenting
; Update
1 (
2 (set-rotation! 0 0 (mult! time 1.5))
3 )
```

Practical example

1. line 1 is read because it's first. Tokenizer sees a '(', which indicates a) the last token we read is over, and b) a new token immediately follows this. So flush (push) the token you were building into our vector as a new entry, and push the '(' so we keep the delimiter.
2. Starting line 2. Repeat step 1. Note we have not made any special logic around the fact we've entered 2 lists. That is not a concern presently, we're just breaking the script up so we can properly parse it later.
3. `set-rotation!` gets read in entirety, then the traversal finds a space, so `set-rotation!` gets pushed as it's own token. No further logic yet around function calls or otherwise.
4. Repeat until third ')'. We should end up with a vector of strings that looks like (, (, set-rotation!, 0, 0, (, mult!, time, 1.5,),),)
5. Now that we have our handy dandy vector of tokens, we need to *do* something with it. Parse the tokens into the actual Node tree
6. Tokens are iterated over again, but with different rules.
    - '(' and ')' now indicate lists, not just a token to save. Each new token is recursively parsed, and if a non list marker is found, it's parsed as either a symbol or a number. No distinction for variables is made in symbols.
    - At the end, you end up with a top level node with a tree to walk down.

Tree representation

```
Top Node: '(' node 1
children: '(' // from line 2, calling this node 2
node 2 children: 'set-rotation!', '0', '0', '(' // oh look another list, clal this node 3
Node 3 children: 'mult!', 'time', '1.5'
```

not depicted: type parsing, but you can figure that out looking at the code pretty well

7. Now the evaluation step. Once again, recursion.

- Switch based on the kind of node you just found. If it's a list, (a '(' we found earlier), evaluate each child iteratively. If you find another list, step down, build args, evalute, then return up.

From our example above:

node 1 starts a new args list. args list gets build with set rotation, 0, 0, and then another '(', so time to recurse down.

Node 2 starts building it's args list, finds mult!, time, and 1.5. list ends, so we hit our 'call' step, for mult! time, and 1.5.

Assume mult! actually works and time is 2, so we get the value of 3, which gets passed up to node2's evaluation.

```
Node then hits call for set-rotation, 0, 0, 3, which assume it does what it's supposed to, t
```